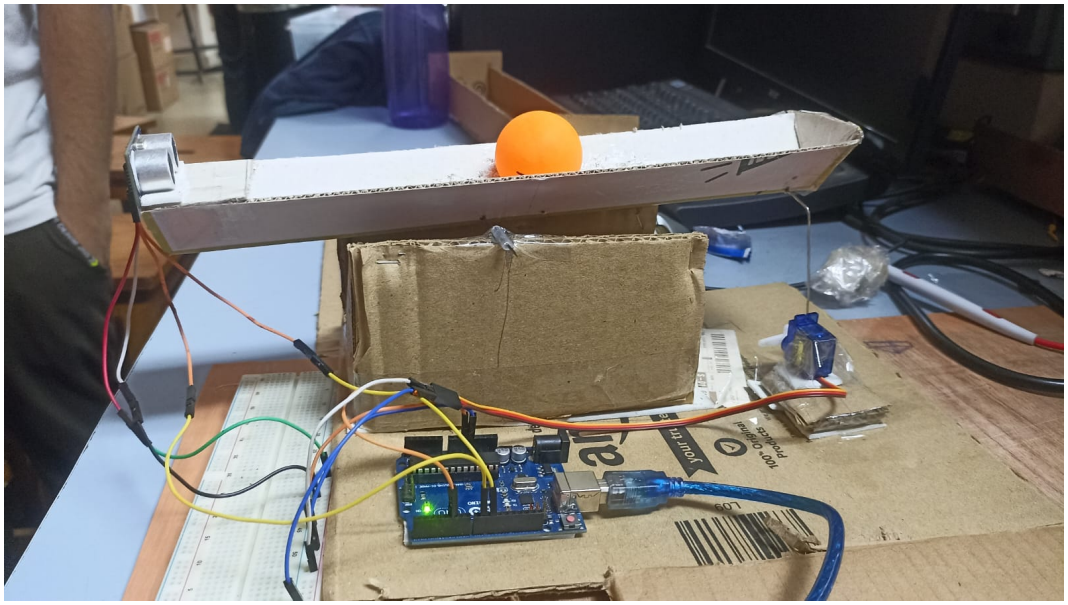
BEAM BALANCE USING PID CONTROLLER

Ball-and-Beam Position Control using Arduino

COURSE LAB PROJECT REPORT



Lalan Kumar Das | Roll No. 230108028

Electrical and Electronics Engineering (EEE)
Indian Institute of Technology Guwahati

Under the Guidance of
Dr. Chayan Bhawal

Course Laboratory Project

. TABLE OF CONTENTS

. PROJECT IDENTITY

Student	Lalan Kumar Das
Roll Number	230108028
Department	Electrical and Electronics Engineering (EEE)
Institute	Indian Institute of Technology Guwahati
Project Type	Course Laboratory Project
Guide	Dr. Chayan Bhawal
Controller	Arduino Uno
Sensors / Actuator	HC-SR04 ultrasonic sensor / Servo motor

1. ABSTRACT

This course-lab project presents a practical Beam Balance (Ball-and-Beam) control system using an Arduino Uno, an HC-SR04 ultrasonic sensor and a servo motor. A lightweight cardboard beam was fabricated to demonstrate closed-loop position control on a real mechanical system.

The ultrasonic sensor measures the ball position. The Arduino compares the measured position with the desired reference and computes a control action. The servo motor changes the beam inclination, which causes the ball to roll under gravity. This creates a feedback loop in which the measured position is continuously used to correct the beam.

The supplied Arduino program contains the P, I and D calculation structure. In the documented code, Kp, Ki and Kd are currently set to zero; therefore, this report describes the implemented PID framework without claiming final tuned performance values.

Keywords: Beam Balance, Ball-and-Beam, Arduino Uno, HC-SR04, Servo Motor, PID Controller, Closed-Loop Control

2. INTRODUCTION AND PROJECT MOTIVATION

The ball-and-beam system is a classical feedback-control experiment. The ball can move along the beam, while a servo motor changes the beam angle. When the beam is tilted, gravity causes the ball to roll. The control objective is to manipulate the beam so that the ball approaches and remains near a selected position.

The laboratory reference describes the ball as having one degree of freedom along the beam and emphasizes rigid mechanical construction, correct sensor alignment and proper controller tuning. The practical prototype developed for this project uses cardboard for the mechanical structure and Arduino for the controller implementation.

The project connects control theory with embedded hardware: the ball position is the controlled variable, the HC-SR04 provides feedback, the servo acts as the actuator, and the beam-and-ball assembly is the physical plant.

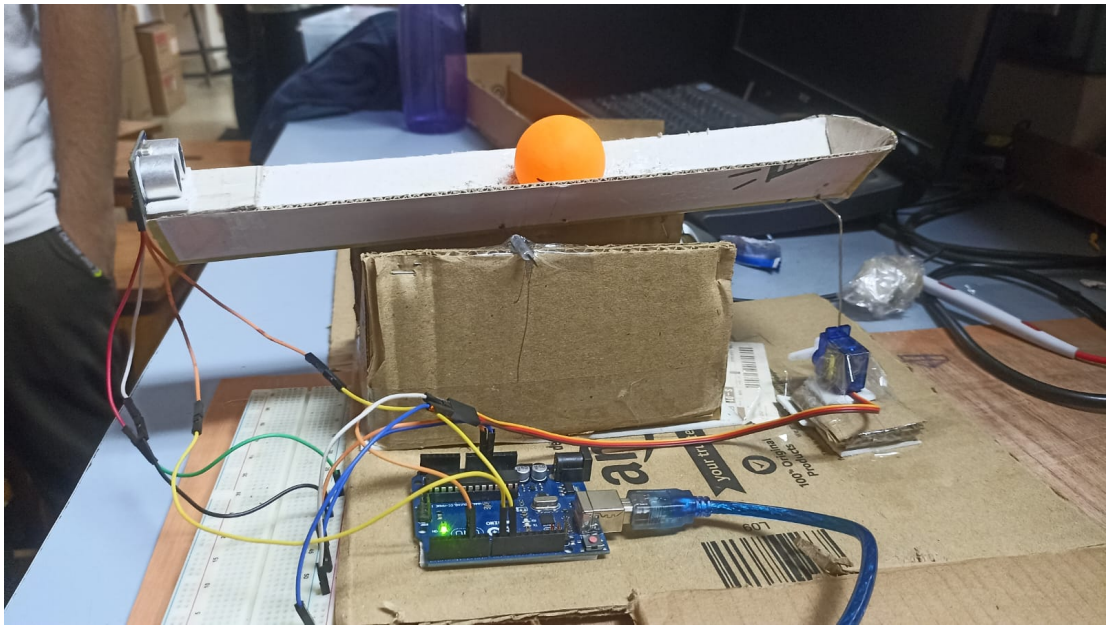


Figure 1. Completed Beam Balance prototype developed for the course-lab project.

3. OBJECTIVES

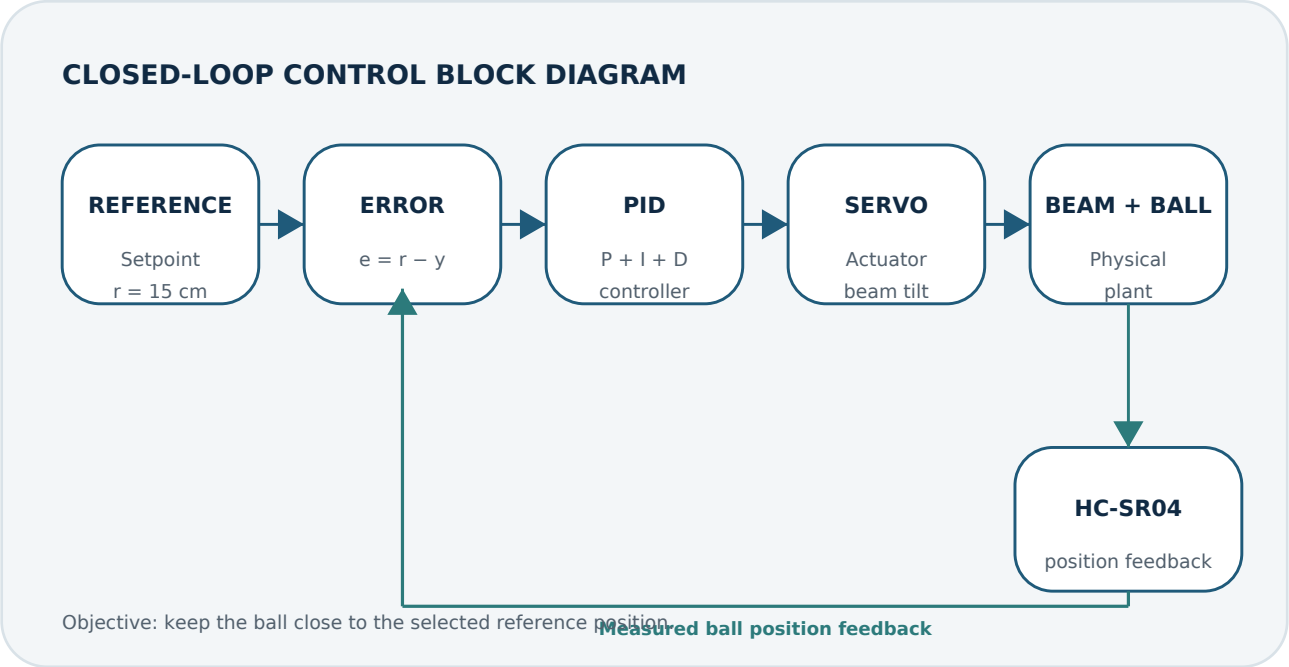
- Construct a functional Beam Balance prototype using a low-cost mechanical structure.
- Measure the ball position using an HC-SR04 ultrasonic sensor.
- Use Arduino Uno for real-time feedback-control computation.
- Drive a servo motor to change the beam inclination.
- Implement the error calculation and PID-control structure in Arduino.
- Study the effect of proportional, integral and derivative actions during tuning.
- Understand practical issues such as sensor noise, mechanical vibration and actuator limitations.

Reference position used in the supplied code: setP = 15 cm

The laboratory module recommends starting with proportional control, observing the response and then deciding whether integral and derivative action are required. It also notes that ultrasonic measurements may need filtering because the sensor data may not be steady.

4. SYSTEM DESCRIPTION AND BLOCK DIAGRAM

The system is a closed-loop position-control system. The Arduino compares the desired ball position with the HC-SR04 measurement, computes the controller output and commands the servo to change the beam inclination.



Working sequence: measure position → calculate error → compute PID action → move servo → correct ball position.

The cycle repeats continuously so that the ball remains close to the selected reference position.

5. HARDWARE DESIGN AND COMPONENTS

Component	Purpose
Arduino Uno	Reads the sensor, executes the control algorithm and commands the servo.
HC-SR04	Measures the ball position from the echo time of an ultrasonic pulse.
Servo motor	Changes the beam inclination through the mechanical linkage.
Beam and support	Provides the physical path and pivot for the ball.
Lightweight ball	Controlled object whose position is measured.
Jumper wires / breadboard	Electrical interconnection between the controller, sensor and actuator.

Design considerations

• The beam should be mounted rigidly so that unwanted horizontal motion and vibration are minimized.

• The ultrasonic sensor should be aligned consistently with the ball path.

• The servo linkage should move the beam smoothly without excessive mechanical play.

• The physical prototype should be lightweight enough for the servo to control reliably.

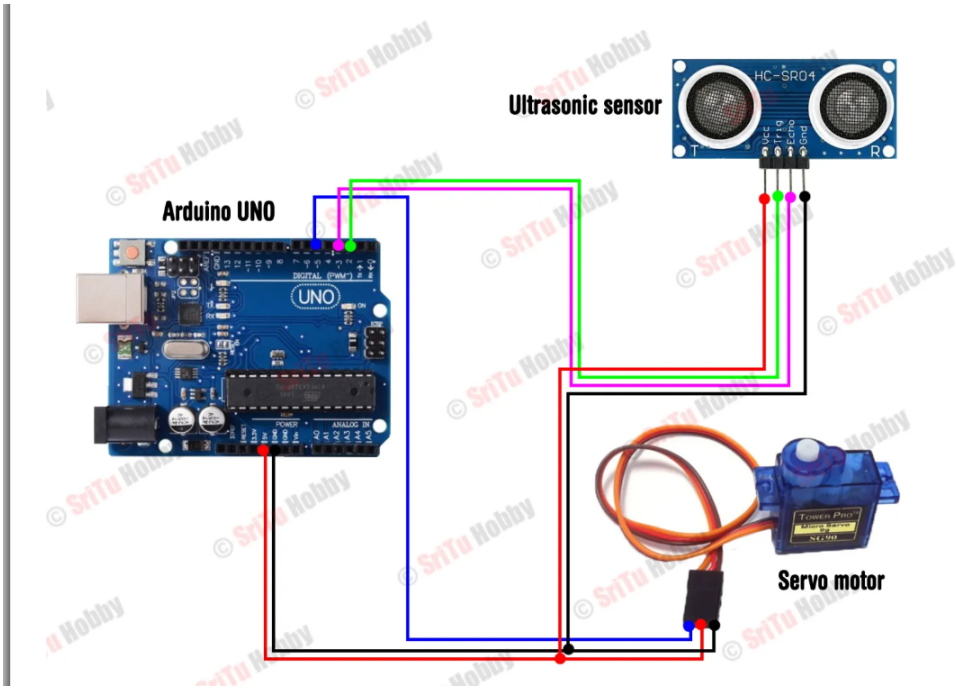


Figure 2. Arduino–HC-SR04–servo wiring arrangement used as the hardware reference.

6. CIRCUIT AND PIN CONFIGURATION

Device	Pin / Signal	Arduino connection
HC-SR04	TRIG	D2
HC-SR04	ECHO	D3
Servo	Control signal	D5
HC-SR04 / Servo	VCC	5 V supply
HC-SR04 / Servo	GND	Common ground

Ultrasonic distance calculation

The supplied program triggers the HC-SR04, measures the echo pulse duration and converts the measured time to distance using:

```
distance (cm) = t / 29 / 2
```

The measured distance is then used as the feedback variable in the error calculation.

7. CONTROL ALGORITHM AND PID FRAMEWORK

Error calculation

e = setpoint − measured position

For the supplied code, the setpoint is 15 cm. The controller forms three terms:

Term	Expression in the supplied code	Meaning
P	error × Kp	Responds to the present error.
I	accumulated error × Ki	Accounts for accumulated error.
D	(error − previous error) × Kd	Responds to the change in error.

Controller output

PIDvalue = Pvalue + Ivalue + Dvalue

The calculated value is converted to an integer, mapped from −135…135 to a servo command range of 135…0, limited to the range 0…135, and then sent to the servo using servo.write().

Important implementation status: Kp = 0, Ki = 0 and Kd = 0 in the supplied code. Therefore, the code contains the PID framework, but final controller tuning and performance claims should only be made after experimental tuning.

Recommended tuning approach

- Begin with P control and observe the response.
- Increase Kp carefully until the response is sufficiently fast without sustained oscillation.
- Add Ki if a steady-state error remains.
- Add Kd only if it improves the transient response and damping.
- Filter noisy ultrasonic measurements when required.

8. ARDUINO IMPLEMENTATION

```
#include <Servo.h>

Servo servo;
#define trig 2
#define echo 3
#define kp 0
#define ki 0
#define kd 0

double priError = 0;
double toError = 0;

void setup() {
  pinMode(trig, OUTPUT);
  pinMode(echo, INPUT);
  servo.attach(5);
  Serial.begin(9600);
  servo.write(50);
}

void loop() {
  PID();
}

long distance() {
  digitalWrite(trig, LOW);
  delayMicroseconds(4);
  digitalWrite(trig, HIGH);
  delayMicroseconds(10);
  digitalWrite(trig, LOW);

  long t = pulseIn(echo, HIGH);
  long cm = t / 29 / 2;
  return cm;
}

void PID() {
  int dis = distance();
  int setP = 15;
  double error = setP - dis;

  double Pvalue = error * kp;
  double Ivalue = toError * ki;
  double Dvalue = (error - priError) * kd;
  double PIDvalue = Pvalue + Ivalue + Dvalue;

  priError = error;
  toError += error;
  Serial.println(PIDvalue);

  int Fvalue = (int)PIDvalue;
  Fvalue = map(Fvalue, -135, 135, 135, 0);

  if (Fvalue < 0) Fvalue = 0;
  if (Fvalue > 135) Fvalue = 135;

  servo.write(Fvalue);
}
```

The program executes the complete feedback cycle in the loop: measure position → calculate error → calculate PID terms → map the controller output → command the servo. Serial.println() is used to observe the controller output during testing.

9. EXPERIMENTAL SETUP, OBSERVATIONS AND LIMITATIONS

The theoretical ball-and-beam concept was converted into a practical cardboard prototype. The beam is supported around a pivot, the servo provides the tilt action, the HC-SR04 measures position, and the Arduino performs the control computation.

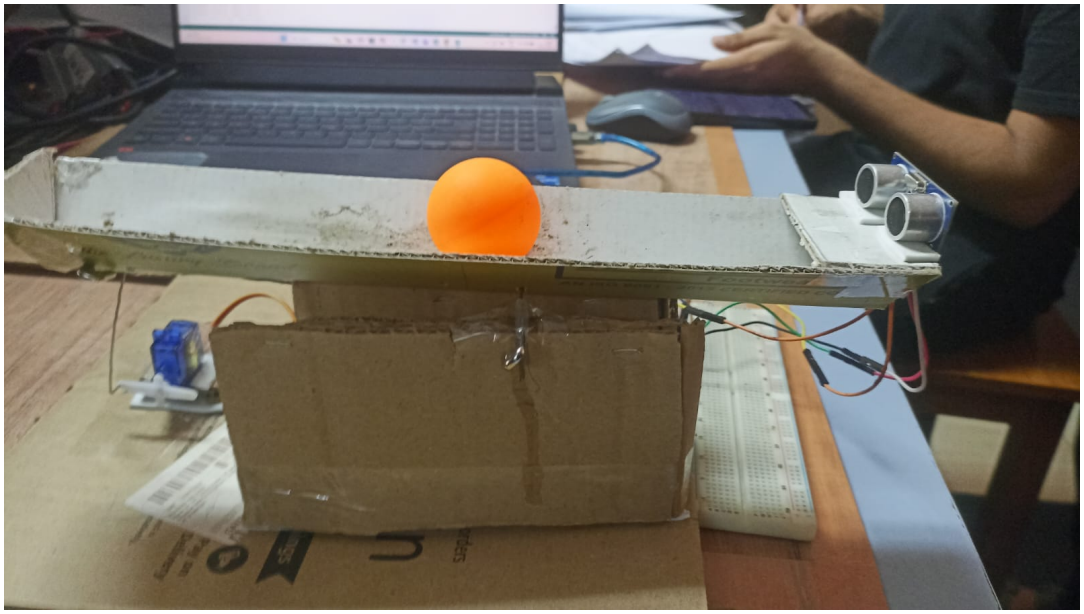


Figure 3. Prototype view showing the beam, ball, sensor, servo linkage and Arduino.

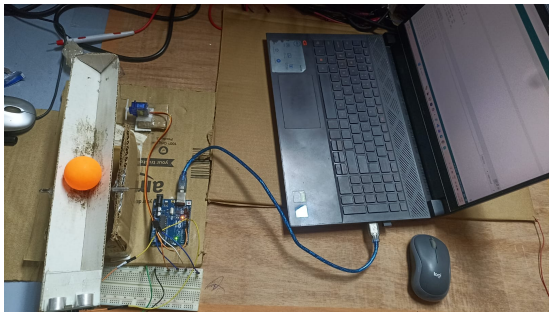


Figure 4. Additional prototype view.

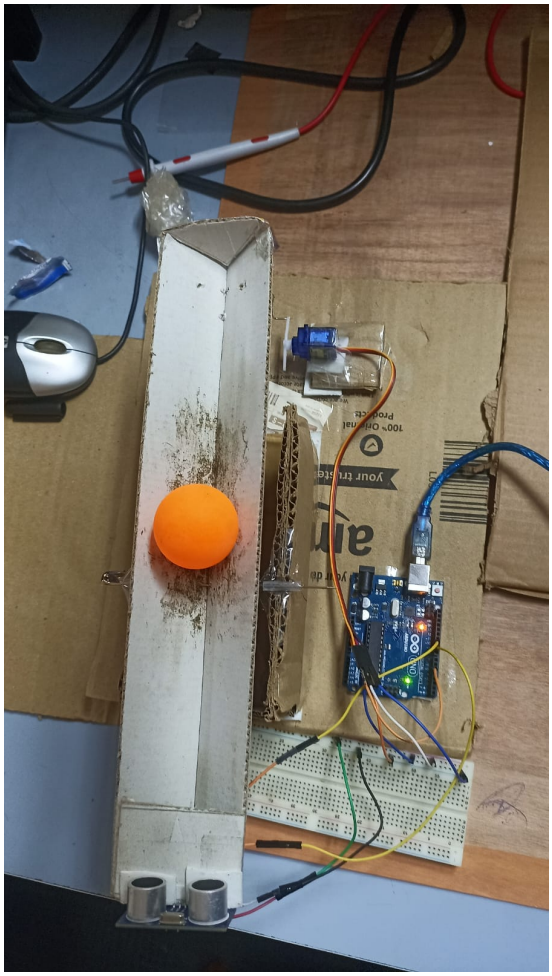


Figure 5. Mechanical and sensor arrangement.

Key observations / limitations

- Cardboard construction can introduce flexibility and vibration.
- Ultrasonic readings can fluctuate because of sensing geometry and surface effects.
- The servo has finite speed and angular resolution.
- The supplied code does not include a measurement filter.
- The supplied code uses zero PID gains, so tuned performance metrics are not reported here.

Useful experimental records

- Record position versus time from the Serial Plotter.
- Compare the response for different Kp values with Ki = Kd = 0.
- Record the effect of adding Ki and Kd.
- If filtering is implemented, compare raw and filtered sensor data.
- Record rise time, settling time, overshoot and steady-state error after final tuning.

10. CONCLUSION AND FUTURE IMPROVEMENTS

A practical Beam Balance using PID Controller was developed as a course laboratory project by Lalan Kumar Das (230108028) under the guidance of Dr. Chayan Bhawal. The project combines sensing, embedded computation, actuation and mechanical prototyping into one closed-loop control system.

The implemented Arduino program establishes the essential feedback structure: the HC-SR04 measures the ball position, the Arduino calculates the position error and PID terms, and the servo changes the beam inclination. The physical prototype demonstrates how the control command influences the motion of the ball and how feedback is used for correction.

The current code documents the PID framework with Kp, Ki and Kd set to zero. The next stage is therefore experimental tuning and validation rather than claiming a final optimized controller.

Future improvements

- Tune Kp, Ki and Kd experimentally using the measured response.
- Apply a moving-average or Kalman filter if ultrasonic data is noisy.
- Use a more rigid beam and stronger pivot/linkage to reduce vibration.
- Improve sensor alignment and mechanical repeatability.
- Provide a stable servo supply with a common ground.
- Plot the measured position and controller output for quantitative analysis.

ACKNOWLEDGEMENT

I sincerely thank Dr. Chayan Bhawal for his guidance and support during this course-lab project. The work provided practical exposure to feedback control, sensor interfacing, servo actuation, Arduino programming and physical system design.

— End of Report —